GRU-Based Workload Prediction for Hybrid Cloud Architectures: Synthetic Training vs. Real-World Generalization

Anonymous Submission
Double-blind review version
Authors and affiliations omitted

Abstract—Accurate short-horizon workload prediction is essential for proactive resource scaling in cloud environments. This paper evaluates a Gated Recurrent Unit (GRU) neural network for 30-second-ahead HTTP traffic forecasting, trained on synthetic workload patterns and validated against real-world HTTP traces (ClarkNet and Calgary). On synthetic test data, the GRU achieves 6.01% RMSE and 4.91% MAE, outperforming six statistical baselines by 46% in RMSE. When transferred to real HTTP traces, the model achieves 17.78% RMSE at 5-minute aggregation on ClarkNet—a 3× degradation relative to synthetic performance—yet still maintains a 3.5× improvement over the best statistical baseline. We identify three factors driving this synthetic-to-real generalization gap: distribution shift between training and test periods, irregular burst patterns absent from synthetic data, and dataset sparsity (Calgary). We further demonstrate the GRU's integration into a hybrid Kubernetes–serverless routing system, achieving 40 ms inference latency with meaningful confidence scores (0.72–0.88). These results establish both the capabilities and the limitations of GRU-based prediction for real-time cloud traffic routing, providing practical guidance for deploying deep-learning-based autoscaling in production.

Index Terms—workload prediction, GRU, cloud computing, time-series forecasting, Kubernetes, serverless, autoscaling

I. Introduction

Modern cloud applications face highly dynamic workloads driven by user behavior patterns, scheduled events, and unexpected traffic spikes. The two dominant deployment paradigms—container orchestration (Kubernetes) and serverless computing (e.g., Knative)—each provide autoscaling mechanisms, but both operate reactively: the Kubernetes Horizontal Pod Autoscaler (HPA) adjusts replica counts based on observed CPU utilization [1], while serverless platforms scale based on concurrency metrics. This reactive approach creates a temporal gap between demand onset and capacity availability, during which Service Level Objective (SLO) violations may occur.

Predictive autoscaling—using machine learning to forecast future workload and pre-provision resources—has been proposed to close this gap [2], [3]. Recurrent neural networks, particularly Long Short-Term Memory (LSTM) [4] and Gated Recurrent Unit (GRU), have demonstrated effectiveness for time-series prediction due to their ability to capture temporal dependencies. GRU offers a favorable trade-off: comparable accuracy to LSTM with ∼25% fewer parameters, faster training, and lower inference latency [5]—attributes critical for real-time routing decisions where inference must complete within tens of milliseconds.

However, a fundamental challenge in deploying deep-learning-based predictors is the training data gap: controlled synthetic data enables supervised training with known patterns, but real-world cloud workloads exhibit non-stationarity, irregular bursts, and distribution shifts that synthetic generators cannot fully capture. This raises a critical question: how well does a GRU trained on synthetic data generalize to real-world HTTP traffic?

This paper addresses this question through the following contributions:

1) A GRU workload prediction model designed for 30-second-ahead HTTP traffic forecasting, achieving 6.01% RMSE on synthetic data with 40 ms inference latency.
2) A comprehensive evaluation across synthetic and real-world datasets (ClarkNet, Calgary), including comparison against six statistical baselines, revealing a 3× synthetic-to-real accuracy gap while maintaining 3.5× improvement over baselines on real data.
3) An analysis of the generalization gap, attributing it to distribution shift, irregular bursts, and dataset sparsity, providing actionable insights for improving real-world deployment of predictive autoscaling.
4) A system integration demonstration showing the GRU feeding into an SLO-aware routing controller for hybrid Kubernetes–serverless traffic management.

II. Related Work

A. Workload Prediction for Cloud Systems

Time-series forecasting for cloud workload prediction has been extensively studied. Classical approaches include autoregressive integrated moving average (ARIMA) models, exponential smoothing, and moving average methods. While effective for stationary or slowly-varying workloads, these methods struggle with the bursty, non-linear patterns typical of HTTP traffic [2].

Deep learning approaches have shown superior performance. LSTM was applied to cloud resource utilization

prediction, demonstrating significant accuracy improvements over ARIMA. Yuan and Liao [6] proposed a time-series-based approach to elastic Kubernetes scaling using Prophet and LSTM. Jegannathan et al. [7] applied time-series forecasting to minimize cold start time in serverless platforms. Vahidnia et al. [8] used reinforcement learning to mitigate cold start problems in serverless computing.

GRU-based prediction has gained traction due to its computational efficiency. Chung et al. [5] empirically demonstrated GRU's comparable performance to LSTM across sequence modeling tasks. For cloud-specific applications, Mondal et al. [3] compared LSTM, Bi-LSTM, and GRU for Kubernetes load prediction, confirming GRU achieves comparable accuracy with simpler training and fewer resource requirements.

B. Elastic Scaling and Hybrid Architectures

The ElaX framework [9] introduced a two-layer elastic provisioning architecture separating workload prediction from resource management. Google's Autopilot [10] uses historical analysis for resource recommendation but remains reactive to recent observations. Beni et al. [11] combined techniques to accelerate Kubernetes scaling, effectively minimizing startup delays. Zhang et al. [12] improved resource efficiency via workload colocation for massive Kubernetes clusters. Serverless platforms have been characterized by Yu et al. [13], who benchmarked cold start latency across major platforms.

Hybrid architectures combining container orchestration with serverless have received less attention. Our work extends ElaX by (1) substituting LSTM with GRU for computational efficiency, (2) extending routing decisions from intra-platform replica scaling to cross-platform traffic distribution, and (3) integrating prediction into a priority-based SLO-aware action hierarchy.

C. Benchmark Datasets

HTTP access logs serve as standard benchmarks for workload prediction. The ClarkNet and Calgary traces were used by Yang et al. [9] in developing the traffic predictor component of the ElaX scaling method. Mondal et al. [3] used the Google Cluster 2011 dataset for Kubernetes scaling prediction. This work uses ClarkNet and Calgary for validation of GRU generalization.

III. Methodology

A. GRU Model Architecture

The GRU model uses a two-layer architecture with 128 hidden units per layer, dropout regularization ($p = 0.2$), followed by a fully connected layer mapping $64 \to 1$. The update and reset gate equations are:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \tag{1}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \tag{2}$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \tag{3}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{4}$$

where z_t is the update gate, r_t is the reset gate, $\tilde{h}_t$ is the candidate activation, and h_t is the output hidden state at time step t.

Training uses the Adam optimizer with learning rate 5×10^{-4}, batch size 32, and early stopping with patience of 25 epochs (maximum 200). The input window length is 60 time steps; the prediction horizon is 30 seconds ahead. The dataset is split 70/15/15 into training, validation, and test sets using temporal ordering to prevent leakage.

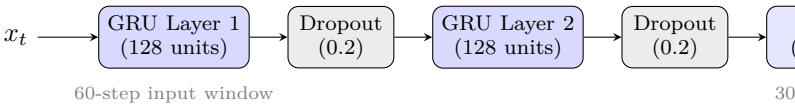

Fig. 1. GRU model architecture: two-layer GRU (128 hidden units each) with dropout and a fully connected output layer. Input: 60 time steps of RPS; output: predicted RPS 30 seconds ahead.

B. Training Data: Synthetic Workload Patterns

The GRU is trained on synthetic traffic patterns combining four components:

- Diurnal cycles: Sinusoidal patterns simulating daily usage peaks and troughs.
- Bursty spikes: Short-duration surges at random intervals.
- Gradual ramps: Linearly increasing segments representing organic growth.
- Baseline noise: Gaussian fluctuations simulating measurement variability.

These components are superimposed and scaled to produce RPS time series. Synthetic training provides (1) controlled labels, (2) arbitrary dataset sizes, and (3) explicit inclusion of ramps and spikes not consistently present in historical traces. A fixed random seed ensures reproducibility.

C. Validation Data: Real HTTP Traces

Two real HTTP trace datasets are used for validation only (not training):

ClarkNet: ∼2M requests from the ClarkNet WWW server (August–September 1995), mean 3.27 RPS per second. Aggregated to 1-minute, 5-minute, and 10-minute intervals.

Calgary: ∼2M requests from the University of Calgary CS department (October 1994), mean 1.20 RPS per second. Very sparse, with extended idle periods.

Using real traces for validation rather than training avoids learning historical idiosyncrasies and preserves strict separation between training data (synthetic) and

external validation (real). Traces are processed via: raw HTTP logs → CLF parsing → timestamp extraction → aggregation → RPS time series → sliding windows.

D. Evaluation Metrics

Prediction accuracy is measured using:

- RMSE%: Root Mean Square Error normalized by traffic range.
- MAE%: Mean Absolute Error normalized by traffic range.
- MAPE: Mean Absolute Percentage Error.

Operational metrics include inference latency (target $<$ 50 ms) and prediction confidence scores.

E. Statistical Baselines

Six baselines are evaluated on the same test data:

1) Moving Average (window = 5)
2) Moving Average (window = 15)
3) Exponential Moving Average ($\alpha = 0.3$)
4) Linear Regression
5) Naïve (last value)
6) Seasonal Naïve (1-hour period)

IV. Experimental Results

A. Synthetic Data Performance

On the synthetic test set, the GRU model meets all predefined accuracy targets (Table I).

TABLE I
GRU Accuracy on Synthetic Test Data

Metric	Value	Target	Status
RMSE%	6.01%	<10%	Met
MAE%	4.91%	<5%	Met
MAPE	∼4.91%	—	Estimated
Mean Load	100.0 RPS	—	Normalization ref.

The RMSE of 6.01% represents prediction error relative to the normalized traffic range, well within the 10% threshold. The MAE of 4.91% indicates predictions deviate by less than 5 RPS on average at 100 RPS mean load.

B. Baseline Comparison on Synthetic Data

Table II compares the GRU against six statistical baselines on the same synthetic test set.

TABLE II
Model Comparison — Synthetic Workload Data

Model	RMSE%	MAE%	MAPE
GRU	6.01	4.91	4.91
Moving Avg (w=15)	11.12	9.36	9.36
EMA (α=0.3)	11.28	9.57	9.57
Moving Avg (w=5)	11.52	9.86	9.86
Linear Regression	11.56	9.77	9.77
Naïve (last value)	14.83	12.74	12.74
Seasonal Naïve (1h)	18.52	15.47	15.47

The GRU achieves approximately 46% lower RMSE% than the best statistical baseline (Moving Average w=15 at 11.12%). All baselines exceed the 10% RMSE target, confirming that simple time-series methods are insufficient for the prediction accuracy required by real-time routing.

C. Real Trace Performance

To assess generalization, the GRU is retrained on ClarkNet and Calgary traces and evaluated on held-out test periods.

TABLE III
GRU Accuracy on ClarkNet Traces

Resolution	RMSE%	MAE%	MAPE	Test N
1-min	26.64	21.27	30.88	1,452
5-min ⋆	17.78	14.48	18.74	243
10-min	17.92	14.81	18.99	116

⋆ Best configuration by RMSE%.

TABLE IV
GRU Accuracy on Calgary Traces

Resolution	RMSE%	MAE%	MAPE	Test N
1-min (14d)	292.53	123.16	72.59	2,964
5-min (30d)	112.74	81.23	181.79	1,236

The best real-trace performance is on ClarkNet at 5-minute aggregation: RMSE% = 17.78%, MAE% = 14.48%, MAPE = 18.74%. This does not meet the original synthetic-calibrated targets (<10% RMSE, <5% MAE), representing a performance gap of approximately 3× compared to synthetic data. Calgary proves unsuitable due to extreme sparsity (mean ∼1 RPS per minute), resulting in metrics dominated by near-zero intervals.

D. Synthetic vs. Real Generalization Gap

Table V quantifies the performance degradation from synthetic to real data.

TABLE V
Synthetic vs. Real Trace Performance Comparison

Metric	Synthetic	ClarkNet 5-min	Ratio
RMSE%	6.01%	17.78%	2.96×
MAE%	4.91%	14.48%	2.95×
MAPE	∼4.91%	18.74%	3.81×

Despite this gap, the GRU substantially outperforms statistical baselines on ClarkNet: 18.74% MAPE versus approximately 65% MAPE for the best baseline (Moving Average), representing a 3.5× improvement. This indicates the GRU captures meaningful temporal patterns even on out-of-distribution data.

E. Live Inference Performance

During live system operation, the GRU prediction server demonstrates operational characteristics suitable for real-time routing (Table VI).

TABLE VI
GRU Live Inference Metrics

Metric	Value	Target	Status
Inference Latency	∼40 ms	<50 ms	Met
Confidence Range	0.72–0.88	>0.6	Met

The 40 ms inference latency confirms the model operates within the constraint for real-time routing decisions. Confidence scores (0.72–0.88) provide a meaningful gating mechanism: the routing controller uses these to modulate reliance on predictions, triggering predictive actions only when confidence exceeds 0.5.

F. Impact on Cold-Start Variance

When integrated into the hybrid system, the prediction pipeline's effect on tail-latency variance was measured via variance decomposition. Cold-start and routing overhead (excess variance beyond baseline scenarios) was isolated for reactive (S3) and predictive (S4) hybrid modes.

For the reactive scenario (S3), 58.2% of total tail-latency variance ($\sigma^2 = 88{,}171$) was attributable to cold-start and routing overhead. For the predictive scenario (S4), this dropped to 23.0% ($\sigma^2 = 47{,}870$)—a 35 percentage-point reduction. Despite similar SCALE_OUT counts (17–18 per run), the prediction pipeline produces substantially less excess variance, suggesting that confidence-modulated weight adjustments create more stable routing behavior even without explicitly triggering the PREDICTIVE action.

The cold-start penalty itself was measured at 682–1,235 ms (mean 959 ms) for Knative pod initialization, compared to 133 ms warm-state p99. This represents an 826 ms overhead above warm-state baseline. In production deployments, setting minScale=1 on Knative would eliminate this penalty entirely, though at the cost of eliminating scale-to-zero savings during idle periods.

V. System Integration

The trained GRU model is integrated into a hybrid Kubernetes–serverless architecture (Fig. 2) via an SLO-aware routing controller. The system uses three layers:

Prediction Layer: The GRU server receives real-time traffic metrics (request rate, latency) and outputs a 30-second-ahead forecast with a confidence score in the range [0, 1]. The confidence is derived from the variance of recent predictions—low variance indicates high confidence.

Control Layer: The routing controller implements a priority-based action hierarchy (Algorithm 1) evaluated every 15 seconds. Four actions are ordered by priority:

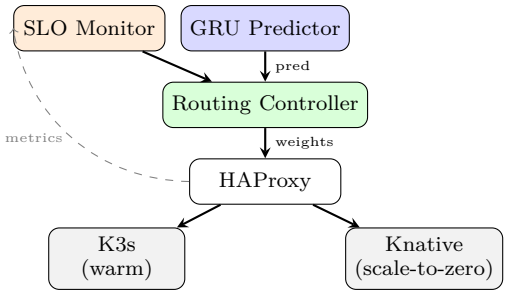

Fig. 2. Hybrid system architecture. The GRU feeds 30-second predictions; the routing controller adjusts HAProxy weights between K3s and Knative based on SLO monitoring and predictions.

1) SCALE_OUT ($p99 > 200$ ms): Shift traffic to serverless in 10-point increments (100/0 → 90/10 → … → 50/50).
2) PREDICTIVE ($p99 < 140$ ms & predicted ≥30% increase): Maintain serverless engagement proactively.
3) OPTIMIZE_COST (stable & underutilized): Reclaim serverless capacity.
4) MAINTAIN (default): No action.

Execution Layer: HAProxy distributes HTTP traffic between K3s (always-warm pods) and Knative (scale-to-zero, burst handling) according to dynamic weights adjusted via the HAProxy Runtime API.

A. Routing Controller Algorithm

The routing controller evaluates system state every 15 seconds and selects an action based on the priority hierarchy. Algorithm 1 shows the decision logic.

Algorithm 1: Routing Controller Decision
Input: $p99$, GRU prediction $\hat{y}$, confidence c, current weights w
repeat every 15s:
if $p99 > 200$ms then SCALE_OUT: shift w toward serverless
else if $p99 < 140$ms and $\hat{y}$ predicts ≥30% ↑ and $c > 0.5$ then PREDICTIVE: maintain serverless weights
else if stable and underutilized then OPTIMIZE_COST
else MAINTAIN
until forever

Fig. 3. Routing controller decision logic.

In a ramp-load validation experiment, the PREDICTIVE action triggered at $p99 = 146$ ms (below SLO threshold) when the GRU predicted a 47% load increase with 72% confidence—demonstrating proactive serverless engagement before any SLO violation occurred. The system correctly shifted traffic weights from 100/0 (pure K8s) through five SCALE_OUT steps to 50/50 (maximum serverless engagement) as load increased, then triggered PREDICTIVE to maintain readiness during the predicted surge.

VI. Discussion

A. Causes of the Synthetic-to-Real Gap

Three factors explain the 3× performance degradation:

Distribution shift: The ClarkNet training period mean was 864 RPS (weekday traffic) while the test period mean was 574 RPS (weekend traffic), representing a distribution shift absent in synthetic data. The model trained on one traffic regime is evaluated on another.

Irregular bursts: Real traces contain burst patterns—sudden request clusters from specific client IPs, flash-crowd events, and multi-modal access patterns—that are poorly represented by the smooth diurnal, burst, and ramp generators used in synthetic training.

Dataset sparsity: The Calgary dataset's academic server traffic follows no clean diurnal cycle and has mean ∼1 RPS per minute. At this density, prediction metrics are dominated by near-zero and zero-value intervals, making meaningful forecasting impossible.

B. Implications for Production Deployment

The results suggest several practical guidelines:

1) Fine-tuning on real data is necessary. Synthetic-trained models capture general temporal patterns but cannot anticipate distribution-specific characteristics. A two-stage approach—pre-train on synthetic, fine-tune on recent production traces—is recommended.
2) 5-minute aggregation is the practical sweet spot. At 1-minute resolution, ClarkNet RMSE% is 26.64%; at 5-minute, it drops to 17.78%. Below 5 minutes, noise dominates signal.
3) Confidence gating prevents harmful predictions. The confidence score (0.72–0.88 observed) enables the routing controller to selectively act on predictions, avoiding proactive scaling during low-confidence forecasts.
4) GRU outperforms baselines even on real data. Despite not meeting synthetic-calibrated thresholds, the GRU's 3.5× improvement over the best statistical baseline confirms it captures transferable temporal patterns.

C. Threats to Validity

Experiments are conducted on a single-node k3d cluster, introducing localhost routing bias that benefits the Kubernetes-only baseline. The GRU server was unavailable during some steady-state experiment batches, preventing PREDICTIVE triggering. Results should be interpreted as mechanism validation rather than production-representative performance benchmarks.

VII. Conclusion

This paper evaluated a GRU-based workload predictor for HTTP traffic forecasting in hybrid cloud architectures. The model achieves 6.01% RMSE on synthetic data, 46% better than the best statistical baseline. On real HTTP traces (ClarkNet at 5-minute aggregation), performance degrades to 17.78% RMSE—a 3× gap attributable to distribution shift, irregular bursts, and dataset sparsity—while maintaining 3.5× improvement over baselines. With 40 ms inference latency and confidence scores in the 0.72–0.88 range, the GRU is suitable for real-time integration into SLO-aware routing controllers.

The synthetic-to-real generalization gap identifies clear directions for improvement: fine-tuning on production traces, adaptive aggregation intervals, and expanded training data incorporating realistic burst patterns. The successful mechanism validation—PREDICTIVE triggering at $p99 = 146$ ms before SLO violation—demonstrates the viability of prediction-integrated hybrid architectures for elastic cloud scalability.

Future work includes multi-node cluster evaluation, comparison with Transformer-based architectures, and adaptive observation windows that adjust to workload dynamics.

Acknowledgment

This work was supported by [anonymized for double-blind review].

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade," Queue, vol. 14, no. 1, pp. 70–93, 2016.
[2] Z. He, "Novel container cloud elastic scaling strategy based on Kubernetes," pp. 1400–1404, 2020.
[3] S. K. Mondal, X. Wu, H. M. D. Kabir, H.-N. Dai, K. Ni, H. Yuan, and T. Wang, "Toward optimal load prediction and customizable autoscaling scheme for Kubernetes," Mathematics, vol. 11, no. 12, 2023.
[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
[5] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," arXiv preprint arXiv:1412.3555, 2014.
[6] H. Yuan and S. Liao, "A time series-based approach to elastic Kubernetes scaling," Electronics, vol. 13, no. 2, p. 285, 2024.
[7] A. P. Jegannathan, R. Saha, and S. K. Addya, "A time series forecasting approach to minimize cold start time in cloud-serverless platform," in 2022 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom), 2022, pp. 325–330.
[8] P. Vahidnia, B. Farahani, and F. S. Aliee, "Mitigating cold start problem in serverless computing: A reinforcement learning approach," IEEE Internet of Things Journal, vol. 10, no. 5, pp. 3917–3927, 2023.
[9] Y. Yang, L. Zhao, Z. Li, L. Nie, P. Chen, and K. Li, "ElaX: Provisioning resource elastically for containerized online cloud services," in Proceedings of the 21st IEEE International Conference on High Performance Computing and Communications, 17th IEEE International Conference on Smart City and 5th IEEE International Conference on Data Science and Systems (HPCC/SmartCity/DSS), 2019, pp. 1987–1994.
[10] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, M. Brand, S. Hungarian, D. Figiela, D. Bajorek, B. Smo'nka, M. Nowak et al., "Autopilot: workload autoscaling at Google," in Proceedings of the 15th European Conference on Computer Systems (EuroSys), 2020, pp. 1–16.
[11] E. H. Beni, E. Truyen, B. Lagaisse, W. Joosen, and J. Dieltjens, "Reducing cold starts during elastic scaling of containers in Kubernetes," in Proceedings of the 36th Annual ACM Symposium on Applied Computing, 2021, pp. 60–68.

[12] X. Zhang, L. Li, Y. Wang, E. Chen, and L. Shou, "Zeus: Improving resource efficiency via workload colocation for massive Kubernetes clusters," IEEE Access, vol. 9, pp. 105 192–105 204, 2021.

[13] T. Yu, Q. Liu, D. Du, Y. Xia, B. Zang, Z. Lu, P. Yang, C. Qin, and H. Chen, "Characterizing serverless platforms with ServerlessBench," in Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC), 2020, pp. 30–44.